Numeric Data Types and Operations

Every data type has a range of values. The compiler allocates memory space for each variable or constant according to its data type.

The following Table lists the numeric data types with their storage sizes and typical ranges.

	Type	Typical Bit Width	Typical Range
Integer Numeric Data Type	int	4bytes	-2147483648 to 2147483647
	unsigned int	4bytes	0 to 4294967295
	signed int	4bytes	-2147483648 to 2147483647
	short int	2bytes	-32768 to 32767
	unsigned short int	2bytes	0 to 65,535
	signed short int	2bytes	-32768 to 32767
	long int	8bytes	-9223372036854775808 to 9223372036854775807
	signed long int	8bytes	same as long int
	unsigned long int	8bytes	0 to 18446744073709551615
	long long int	8bytes	-(2^63) to (2^63)-1
	unsigned long long int	8bytes	0 to 18,446,744,073,709,551,615
Floating-Point Numeric Data Type	float	4bytes	Negative range: -3.4028235E+38 to -1.4E-45 Positive range: 1.4E-45 to 3.4028235E+38
	double	8bytes	Negative range: -1.7976931348623157E+308 to -4.9E-324 Positive range: 4.9E-324 to 1.7976931348623157E+308
	long double	12bytes	Negative range: -1.18E+4932 to -3.37E-4932 Positive range: 3.37E-4932 to 1.18E+4932

C++ uses three types for integers: short, int, and long. Each integer type comes in two flavors: signed and unsigned.

C++ uses three types for floating-point numbers: float, double, and long double. The double type is usually twice as big as float. So, the double is known as double precision, while float is single precision. The long double is even bigger than double. For most applications, using the double type is desirable.

For convenience, C++ defines constants INT_MIN, INT_MAX, LONG_MIN, LONG_MAX, FLT_MIN, FLT_MAX, DBL_MIN, and DBL_MAX in the <limits> header file. These constants are useful in programming. Run the following code and see what constant values are defined by your compiler:

Ex_0301.cpp

```cpp
#include <iostream>
#include <limits>
using namespace std;

int main()
{
   cout << "INT_MIN is " << INT_MIN << endl;
   cout << "INT_MAX is " << INT_MAX << endl;
   cout << "LONG_MIN is " << LONG_MIN << endl;
   cout << "LONG_MAX is " << LONG_MAX << endl;
   cout << "FLT_MIN is " << FLT_MIN << endl;
   cout << "FLT_MIN is " << FLT_MAX << endl;
   cout << "DBL_MIN is " << DBL_MIN << endl;
   cout << "DBL_MIN is " << DBL_MAX << endl;

   return 0;
}
```

When you run the above code it gives the following output

```
INT_MIN is -2147483648
INT_MAX is 2147483647
LONG_MIN is -2147483648
LONG_MAX is 2147483647
FLT_MIN is 1.17549e-38
FLT_MIN is 3.40282e+38
DBL_MIN is 2.22507e-308
DBL_MIN is 1.79769e+308

--------------------------------
Process exited after 0.1496 seconds with return value 0
Press any key to continue . . .
```

The following example displays the size of int, long, and double, and variables age and area on your machine.

Ex_0302.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  cout << "The size of int: " << sizeof(int) << " bytes" << endl;
  cout << "The size of long: " << sizeof(long) << " bytes" << endl;
  cout << "The size of double: " << sizeof(double)
    << " bytes" << endl;

  double area = 5.4;
  cout << "The size of variable area: " << sizeof(area)
    << " bytes" << endl;

  int age = 31;
  cout << "The size of variable age: " << sizeof(age)
    << " bytes" << endl;

  return 0;
}
```

When you run the above code it gives the following output

```
The size of int: 4 bytes
The size of long: 4 bytes
The size of double: 8 bytes
The size of variable area: 8 bytes
The size of variable age: 4 bytes

--------------------------------
Process exited after 0.1867 seconds with return value 0
Press any key to continue . . .
```

Numeric Operators

Operator	*Name*	*Example*	*Result*
+	Addition	34 + 1	35
-	Subtraction	34.0 − 0.1	33.9
*	Multiplication	300 * 30	9000
/	Division	1.0 / 2.0	0.5
%	Modulus	20 % 3	2

Notes:

- When both operands of a division are integers, the result of the division is the quotient and the fractional part is truncated. For example, 5/2 yields 2, not 2.5, and –5/2 yields -2, not –2.5. To perform regular mathematical division, one of the operands must be a floating point number. For example, 5.0/2 yields 2.5.
- The % operator, known as *modulo* or *remainder* operator, works only with integer operands and yields the remainder after division. The left-hand operand is the dividend and the right hand operand the divisor. Therefore, 7%3 yields 1, 3%7 yields 3, 12%4 yields 0, 26%8 yields 2, and 20%13 yields 7.

Example: The following program obtains minutes and remaining seconds from an amount of time in seconds. For example, 500 seconds contains 8 minutes and 20 seconds.

Ex_0303.cpp

```cpp
#include <iostream>
using namespace std;

int main()
{
  // Prompt the user for input
  int seconds;
  cout << "Enter an integer for seconds: ";
  cin >> seconds;
  int minutes = seconds / 60;
  int remainingSeconds = seconds % 60;
  cout << seconds << " seconds is " << minutes <<
    " minutes and " << remainingSeconds << " seconds " << endl;

  return 0;
}
```

```
Enter an integer for seconds: 500
500 seconds is 8 minutes and 20 seconds
```

Exponent Operations

The pow(a, b) function can be used to compute a^b. pow is a function defined in the cmath library. The function is invoked using the syntax pow(a, b) (i.e., pow(2.0, 3)) that returns the result of a^b (2^3). Here, a and b are parameters for the pow function and numbers 2.0 and 3 are actual values used to invoke the function. For example,

```
cout << pow(2.0, 3) << endl; // Display 8.0
cout << pow(4.0, 0.5) << endl; // Display 2.0
cout << pow(2.5, 2) << endl; // Display 6.25
cout << pow(2.5, -2) << endl; // Display 0.16
```

Note that some C++ compilers require that either a or b in pow(a, b) be a decimal value. Here we use 2.0 rather than 2.

Example The last program for computing the area of a circle is modified as follows;

Ex_0304.cpp

```
1  // Compute the Circle Area
2  #include <iostream>
3  #include<cmath>
4  using namespace std;
5
6  int main()
7  {
8      double radius;
9      double area;
10     const double PI=3.14159;
11
12     // Step 1: Read in the radius
13     radius = 20;
14
15     // Step 2: Compute the area
16     // area = radius*radius*3.14159;
17     area = pow(radius,2)*PI;
18
19     // Step 3: Display the area
20     cout << "The area is " << area << endl;
21 return 0;
22 }
```

```
The area is 1256.64
```

Operator Precedence

Writing numeric expressions in C++ involves a straightforward translation of an arithmetic expression using C++ operators. For example, the arithmetic expression

$$\frac{3+4x}{5}-\frac{10(y-5)(a+b+c)}{x}+9\left(\frac{4}{x}+\frac{9+x}{y}\right)$$

can be translated into a C++ expression as

```
(3 + 4 * x) / 5 - 10 * (y - 5) * (a + b + c) / x +
9 * (4 / x + (9 + x) / y)
```

When more than one operator is used in an expression, the following operator precedence rule is used to determine the order of evaluation.

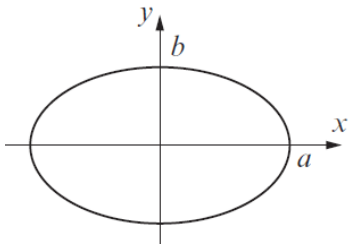
- Multiplication, division, and remainder operators are applied next. If an expression contains several multiplication, division, and remainder operators, they are applied from left to right.
- Addition and subtraction operators are applied last. If an expression contains several addition and subtraction operators, they are applied from left to right.

Example

The perimeter of an ellipse can be approximated by

$$p=\pi\left[3(a+b)-\sqrt{(3a+b)(a+3b)}\right]$$

Calculate the perimeter of an ellipse with $a = 18\ in.$ and $b = 7\ in.$



Ex_0305.cpp

```
// Compute the perimeter of an ellipse
#include <iostream>
#include<cmath>
using namespace std;

int main()
{
    double a , b;
    double perimeter;
    const double PI=3.14159;

    cout << "Enter a and b: ";
    cin >> a >> b;
    perimeter = PI*(3*(a+b)-sqrt((3*a+b)*(a+3*b)));
    cout << "The perimeter is " << perimeter << endl;
return 0;
}
```

```
Enter a and b: 18 7
The perimeter is 82.3882
```

Augmented Assignment Operators

The operators +, -, *, /, and % can be combined with the assignment operator to form augmented operators. The following table shows how augmented operators work.

Operator	*Name*	*Example*	*Equivalent*
+=	Addition assignment	`i += 8`	`i = i + 8`
-=	Subtraction assignment	`i -= 8`	`i = i - 8`
*=	Multiplication assignment	`i *= 8`	`i = i * 8`
/=	Division assignment	`i /= 8`	`i = i / 8`
%=	Modulus assignment	`i %= 8`	`i = i % 8`

Increment and Decrement Operators

The increment (++) and decrement (--) operators are for incrementing and decrementing a variable by 1.

The ++ and -- are two shorthand operators for incrementing and decrementing a variable by 1.

For example, the following code increments i by 1 and decrements j by 1.

```
int i = 3, j = 3;
i++; // i becomes 4
j--; // j becomes 2
```

This is called *postincrement* or *postdecremen*

A *preincrement* and *predecrement* are also possible

```
int i = 3, j = 3;
++i; // i becomes 4
--j; // j becomes 2
```

Numeric Type Conversions

Floating-point numbers can be converted into integers using explicit casting.

Can you assign an integer value to a floating-point variable? Yes.

Can you assign a floating point value to an integer variable? Yes.

When assigning a floating-point value to an integer variable, the fractional part of the floating-point value is truncated (not rounded). For example:

```
int i = 34.7;       // i becomes 34
double f = i;       // f is now 34
double g = 34.3;    // g becomes 34.3
int j = g;          // j is now 34
```

Can you perform binary operations with two operands of different types? Yes.

If an integer and a floating-point number are involved in a binary operation, C++ automatically converts the integer to a floating-point value. So, 3*4.5 is the same as 3.0*4.5.

C++ also allows you to convert a value from one type to another explicitly by using a *casting operator*. The syntax is

```
static_cast<type>(value)
```

Example

Ex_0306.cpp

```
#include <iostream>
using namespace std;

int main() {
    double pi = 3.14159;

    cout << "The value of pi with a type double is " << pi << endl;

    // Explicitly convert double to int (truncating the decimals)
    int truncated_pi = static_cast<int>(pi);
    cout << "The value of pi after applying statc cast to int is "
    << truncated_pi << endl;

    return 0;
}
```

```
The value of pi with a type double is 3.14159
The value of pi after applying statc cast to int is 3
```

Example

Ex_0307.cpp

```cpp
#include <iostream>
using namespace std;

int main() {
    int total_points = 150;
    int items = 16;

    // Wrong: Integer division yields 9
    double bad_avg = total_points / items;
    cout << "Without cast: " << bad_avg << endl;       // Outputs: 9

    //  Correct: Casts 'total_points' to double first, yielding 9.375
    double good_avg = static_cast<double>(total_points) / items;
    cout << "With static_cast: " << good_avg << endl; // Outputs: 9.375

    return 0;
}
```

```
Without cast: 9
With static_cast: 9.375
```

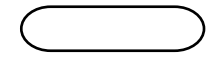
Flowcharts

A flowchart is a visual representation that illustrates the sequential flow of logic or operations within a program or process. It employs symbols and arrows to depict the movement of control between each step. Flowcharts are valuable for designing programs that involve complex computational algorithms. They also help in illustrating and comprehending the sequence of a program's code.

Flowcharts help programmers:

- Plan the logic of a program before coding.
- Visualize how data and decisions move through the program.
- Communicate program logic clearly to others.
- Debug and improve program structure.

Common Flowchart Symbols:



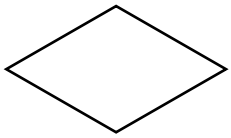
Oval or rounded rectangle marks the beginning or end of a process.



Parallelogram denotes data entry or data output.



Rectangle represents a process step, operation, or instruction.



Diamond shows a branching point where a Yes/No or True/False decision is made.

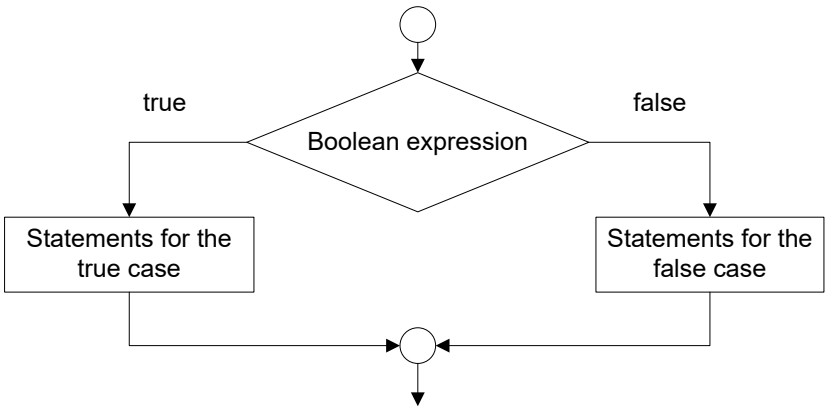


Small circle or labeled connector symbol connects parts of a flowchart that are on different pages or areas.



Arrows show the direction of flow (sequence of steps).

Until the moment, we have introduced elementary programs in which statements are executed sequentially. Suppose, for some reason you do not need to execute the program statements in sequential order. In other words, suppose that the program is composed of different blocks of statements and you need to execute only one block when a certain condition is satisfied.



The term *selection* in C++ programming means executing a specific block of statements according to a certain condition. C++ provides several types of selection statements: one-way if statements, two-way if-else statements, nested if statements, switch statements, and conditional expressions.

Boolean Expression

A boolean expression is a logical statement that evaluates to a single value: true or false. These expressions are fundamental in programming and logic, using operators like AND, OR, and NOT to combine values, variables, or functions to form a condition.

Relational Operators

Operator	*Mathematics Symbol*	*Name*	*Example (radius is 5)*	*Result*
`<`	<	less than	`radius < 0`	`false`
`<=`	≤	less than or equal to	`radius <= 0`	`false`
`>`	>	greater than	`radius > 0`	`true`
`>=`	≥	greater than or equal to	`radius >= 0`	`true`
`==`	=	equal to	`radius == 0`	`false`
`!=`	≠	not equal to	`radius != 0`	`true`

Boolean Operators

Operator	*Name*	*Description*
`!`	not	logical negation
`&&`	and	logical conjunction
`||`	or	logical disjunction